Отчет о лабораторной работе 1

Серышева Юлия,
Камаева Екатерина,
23КНТ5

Архитектура тестовой системы и методология замеров времени

Эксперименты проводились в операционной системе Linux (дистрибутив Ubuntu), функционирующей поверх виртуализированной инфраструктуры WSL2 (Windows Subsystem for Linux 2) на базе гипервизора Hyper-V. Исполнение многопоточных программ Pthreads в таком окружении имеет определенные архитектурные особенности по сравнению с нативными системами:

1. двухуровневое планирование потоков;

Если в нативной системе планировщик Linux напрямую распределяет потоки Pthreads по физическим ядрам, то в WSL2 они сначала распределяются гостевой ОС по виртуальным процессорам, которые затем повторно планируются гипервизором Hyper-V и Windows. Отсюда дополнительные накладные расходы и возможная нестабильность в измерениях;

2. слой виртуализации I/O.

Системные вызовы записи на диск проходят через виртуальную файловую систему WSL2 VHDX, вызывая дополнительные накладные расходы при синхронном вводе-выводе.

Методология хронометрирования

Для фиксации времени исполнения программы использован механизм на основе файла timer.h, использующий системный вызов gettimeofday. Данный

вызов заполняет структуру struct timeval t, содержащую время в секундах tv_sec и микросекундах tv_usec.

Расчет интервала времени ΔT между начальной (t_{start}) и конечной (t_{end}) точками замера осуществляется по формуле:

$$\Delta T = (tv_sec_{end} - tv_sec_{start}) + \frac{tv_usec_{end} - tv_usec_{start}}{10^6}$$

Также первичный прогон программы отбрасывается и не включается в итоговую выборку (так называемый прогрев), а итоговое значение времени вычисляется как среднее арифметическое полученных результатов (5 измерений).

Для автоматизации проведения экспериментов был написан скрипт, рассчитывающий среднее время выполнения работы программы с заданным количеством потоков за 5 запусков и записывающий итоги в сводную таблицу.

Формулы метрик параллельной эффективности

Оценка масштабируемости параллельной программы проводится на основе стандартных критериев ускорения $S(p)$ и эффективности $E(p)$:

$$S(p) = \frac{T_1}{T_p}$$

$$E(p) = \frac{S(p)}{p} = \frac{T_1}{p \cdot T_p}$$

где T_1 - время выполнения алгоритма на одном вычислительном потоке, T_p - время выполнения при использовании p параллельных рабочих потоков.

Задание 1

1. **Математическая модель задачи поиска N точек множества Мандельброта**

Можно доказать, что число c принадлежит множеству Мандельброта тогда и только тогда, когда

$$(\forall n \in N)\,|z_n| < 2,\ \ [\![z]\!]_1 = 0$$

Имеет смысл установить верхнюю границу, до которой будем перебирать n, чтобы не уйти в бесконечный цикл. Положим n = 1000.

Для выбора самих точек для проверки будем выбирать точки на плоскости случайным образом. Границы охватываемой области (1):

x_(min)= -2

y_(min)= -1.25

x_(max)= -0.5

y_(max)= 1.25

Итак, берётся случайная точка с координатами (r_x, r_y) на области (1) и n = 1000 раз проверяется условие принадлежности точки множеству Мандельброта, т.е. на каждом шаге делаем операцию:

$Z_{n+1} = Z_n^2 + c$, где $c = r_x + i\,r_y$ (точка, полученная случайным образом).

$$Z_{n+1} = ({z_x}_n{}^2 - {z_y}_n{}^2 + r_x) + i(2{z_x}_n{z_y}_n + r_y)$$

$${z_x}_{n+1} = ({z_x}^2 - {z_y}^2 + r_x)$$

$${z_y}_{n+1} = (2z_x z_y + r_y)$$

Само условие: $|Z_{n+1}|^2 < 4$ или ${z_x}_{n+1}{}^2 + {z_y}_{n+1}{}^2 < 4$

2. Схема параллельного алгоритма на pthreads

Работать с потоками будем так: каждому потоку необходимо будет сгенерировать определенное количество точек, входящих в множество

Мандельброта. Каждый поток будет заполнять свой определенный диапазон индексов в общих массивах x и y. При этом при записи результатов не будут использоваться мьютексы, поэтому нет нужды в синхронизации результатов.

Подробнее рассмотрим метод routine:

```
void* routine(void* rank){
    long my_rank = (long)rank;

    int q = n_points / thread_count;
    int rem = n_points % thread_count;
    int my_points = q + (my_rank < rem ? 1 : 0);
    int my_first_i = my_rank*q + (my_rank < rem ? my_rank : rem);

    unsigned int my_seed = base_seed + (unsigned int)my_rank*7919u;

    double local_buffer_x[BUFFER_SIZE];
    double local_buffer_y[BUFFER_SIZE];
    int local_count = 0;

    int points_found = 0;

    while(points_found < my_points){
        double r_x = (double)rand_r(&my_seed)/(double)RAND_MAX*(X_MAX - X_MIN) + X_MIN;
        double r_y = (double)rand_r(&my_seed)/(double)RAND_MAX*(Y_MAX - Y_MIN) + Y_MIN;

        double z_x = 0.0;
        double z_y = 0.0;
        int inside = 1;

        for(int counter = 0; counter < COUNTER_MAX; ++counter){
            double t_x = z_x*z_x - z_y*z_y + r_x;
            double t_y = 2.0*z_x*z_y + r_y;
            z_x = t_x;
            z_y = t_y;
            if(z_x*z_x + z_y*z_y > 4.0){
                inside = 0;
                break;
            }
        }

        if(inside){
            local_buffer_x[local_count] = r_x;
            local_buffer_y[local_count] = r_y;
            local_count++;
            points_found++;

            if (local_count == BUFFER_SIZE) {
                memcpy(&x[my_first_i], local_buffer_x, BUFFER_SIZE * sizeof(double));
                memcpy(&y[my_first_i], local_buffer_y, BUFFER_SIZE * sizeof(double));

                my_first_i += BUFFER_SIZE;
                local_count = 0;
            }
        }
    }

    if (local_count > 0) {
        memcpy(&x[my_first_i], local_buffer_x, local_count * sizeof(double));
        memcpy(&y[my_first_i], local_buffer_y, local_count * sizeof(double));
    }

    return NULL;
}
```

Кол-во точек, которые должен вычислить поток, помещается в переменную q. Это целая часть от деления n_points на thread_count. В ситуации, когда невозможно распределить точки по потокам поровну и остаток от деления равен m, к потоку может прибавиться дополнительная

точка, если он входит в первые m потоков. Далее определяются границы массива исходя из ранга потока и кол-ва точек для вычисления.

Для генерации случайных координат используется rand_r() – безопасная версию rand(). Функция не хранит сид глобально, а принимает его в качестве аргумента. Поэтому необходимо инициализировать сид потока, он будет храниться в my_seed. Base_seed – начальный сид (время компьютера), к нему прибавляется ранг, умноженный на простое число (7919 взято произвольно).

Далее запускается цикл до тех пор, пока поток не найдет все точки и не заполнит свою часть массива, берётся случайная точка на области (1) и COUNTER_MAX (1000) раз проверяется условие принадлежности точки множеству Мандельброта ($|Z_{n+1}|^2 < 4$ или ${z_x}_{n+1}^2 + {z_y}_{n+1}^2 < 4$).

Если точка попала в множество, она записывается в локальный массив. При достижении BIFFER_SIZE (64) сгружаем накопленные точки в общий массив. Было решено накапливать точки в маленьком массиве (а не все точки, чтоб не выделять память в куче, т.к. при возникновении проблем с потоком у нас осталась бы висеть неосвобожденная память).

3. Результаты замеров производительности

Сводная таблица производительности

Тестирование проводилось на Intel Core i7-12700H, ОС Ubuntu 26.04.1 LTS.

Компиляция проходила с помощью gcc с флагом оптимизации -O2.

Для замеров производительности используется скрипт benchmark.sh, запись в файл при этом не учитывалась.

Для каждого числа потоков (это 1, 2, 4, 6, 8) программа запускалась 5 раз, бралось среднее всех измерений. Для каждого измерения искались млн. точек.

Потоки	**Среднее время**	**Ускорение**	**Эффективность**
1	3.3096	1.0000	1.0000
2	1.6220	2.0403	1.0201
4	0.8309	3.9827	0.9956
6	0.5441	6.0827	1.0137
8	0.4284	7.7252	0.9656

По результатам можно сказать, что эта задача хорошо параллелится, с увеличением числа потоков время выполнения падает. Ускорение на 8 потоках больше, чем в 7 раз. По поводу эффективности, можно предположить, в диапазоне от 1 до 6 потоков нагрузка распределялась строго по физическим P-ядрам, а при переходе к 8 эффективность немного падает.

Задание 2

1. Математическая модель задачи N тел

Закон всемирного тяготения и 3-й закон Ньютона

Моделирование гравитационного взаимодействия N материальных точек осуществляется в трехмерном пространстве. На каждое тело q со стороны тела k действует гравитационная сила F_{qk}, определяемая по Закону всемирного тяготения:

$$F_{qk} = G\frac{m_q m_k}{|r_k - r_q|^3}(r_k - r_q)$$

где $G = 6.67430 \cdot 10^{-11} Н \cdot м^2/кг^2$ - гравитационная постоянная, m_q и m_k - массы взаимодействующих тел, а r_q и r_k - их радиус-векторы положения в трехмерной системе координат.

Суммарная векторная сила F_q, действующая на тело q со стороны остальных $N-1$ тел, рассчитывается по следующей формуле:

$$F_q = \sum_{k=1, k \neq q}^{N} F_{qk}$$

Согласно 3-му закону Ньютона, силы взаимодействия между парой тел равны по модулю и противоположны по направлению:

$$F_{pq} = -F_{qp}$$

Использование этого свойства позволяет вычислять вектор силы взаимодействия для каждой пары тел ровно один раз. Это сокращает общий объем вычислений парных сил вдвое, с $N(N-1)$ до $N(N-1)/2$ операций, таким образом вычислительная сложность одной итерации снижается.

Метод Эйлера 1-го порядка

Для численного интегрирования системы дифференциальных уравнений движения применяется метод Эйлера 1-го порядка. Пересчет координат (x_q^n, y_q^n, z_q^n) и составляющих скорости $(v_{xq}^n, v_{yq}^n, v_{zq}^n)$ тела q на n-м шаге по времени с фиксированным интервалом Δt выполняется по следующим формулам:

$$x_q^n = x_q^{n-1} + v_{xq}^{n-1}\Delta t \qquad y_q^n = y_q^{n-1} + v_{yq}^{n-1}\Delta t \qquad z_q^n = z_q^{n-1} + v_{zq}^{n-1}\Delta t$$

$$v_{xq}^n = v_{xq}^{n-1} + \frac{F_{xq}^{n-1}}{m_q}\Delta t \qquad v_{yq}^n = v_{yq}^{n-1} + \frac{F_{yq}^{n-1}}{m_q}\Delta t \qquad v_{zq}^n = v_{zq}^{n-1} + \frac{F_{zq}^{n-1}}{m_q}\Delta t$$

2. Схема параллельного алгоритма на pthreads

Декомпозиция вычислений на потоки

Распараллеливание вычислений между потоками pthreads осуществим путем разделения внешнего цикла расчетов парных сил. Массив thread_forces содержит вклад гравитационных сил для выделенного подмножества частиц и соответствующего потока, который производил расчет сил.

Учет 3-го закона Ньютона ($F_{pq} = -F_{qp}$) приводит к тому, что один поток при обработке пары (p, q) должен одновременно обновить составляющие сил для обоих тел.

Для устранения состояния гонки каждый поток рассчитывает вычисленные силы в двумерном массиве thread_forces[nthreads][N]. При таком подходе поток пишет исключительно в свой столбец/строку памяти. Частицы между потоками разделены циклично, это помогает избежать дисбаланса при вычислении локальных сил потоком, и нагрузка на потоки остается равной.

```
void Calculate_forces(long my_rank) {
   int i, j;
   double rx, ry, rz, dist_sq, dist, coef;
   double fx, fy, fz;

   for (i = 0; i < N; i++) {
      thread_forces[my_rank][i].fx = 0.0;
      thread_forces[my_rank][i].fy = 0.0;
      thread_forces[my_rank][i].fz = 0.0;
   }

   for (i = my_rank; i < N - 1; i += nthreads) {
      for (j = i + 1; j < N; j++) {
         rx = particles[j].x - particles[i].x;
         ry = particles[j].y - particles[i].y;
         rz = particles[j].z - particles[i].z;
         dist_sq = rx*rx + ry*ry + rz*rz + EPSILON;

         dist = sqrt(dist_sq);
         coef = G * particles[i].m * particles[j].m / (dist_sq * dist);

         fx = coef * rx;
         fy = coef * ry;
         fz = coef * rz;

         thread_forces[my_rank][i].fx += fx;
         thread_forces[my_rank][i].fy += fy;
         thread_forces[my_rank][i].fz += fz;

         thread_forces[my_rank][j].fx -= fx;
         thread_forces[my_rank][j].fy -= fy;
         thread_forces[my_rank][j].fz -= fz;
      }
   }
}
```

Объединение данных и барьерная синхронизация

```
void* Thread_work(void* rank) {
   long my_rank = (long) rank;
   int start_i, end_i;

   start_i = my_rank * chunk;
   end_i = (start_i + chunk < N) ? start_i + chunk : N;

   for (int step = 0; step <= num_steps; step++) {

      Calculate_forces(my_rank);
      pthread_barrier_wait(&barrier);

      Reduce_forces(start_i, end_i);
      pthread_barrier_wait(&barrier);

      if (my_rank == 0 && out_fp != NULL) {
         fprintf(out_fp, "%.6f", step * time_step);

         for (int i = 0; i < N; i++) {
            fprintf(out_fp, ",%.6f,%.6f,%.6f", particles[i].x, particles[i].y, particles[i].z);
         }
         fprintf(out_fp, "\n");
      }
      pthread_barrier_wait(&barrier);

      Update_particles(start_i, end_i);
      pthread_barrier_wait(&barrier);

   }

   return NULL;
}
```

Можно представить итерацию в цикле по времени в виде нескольких фаз вычислений и синхронизации:

1. Расчет локальных сил Calculate_forces (параллельное вычисление парных взаимодействий частиц потоками $pthreads$ с накоплением результатов в независимые локальные буферы thread_forces)
2. Синхронизация (точка останова pthread_barrier_wait, гарантирующая завершение вычисления всех парных сил всеми рабочими потоками перед началом объединения).
3. Редукция сил Reduce_forces (параллельное суммирование накопленных вкладов сил из локальных буферов всех потоков в результирующий глобальный массив сил global_forces. Каждому потоку назначается диапазон индексов частиц $[i_{start}, i_{end}]$, для

которых он суммирует компоненты $global_forces[i] = \sum_{k=0}^{p-1} thread_forces[k][i]$, что полностью исключает конфликты записи)

```
void Reduce_forces(int start_i, int end_i) {
    int i, t;

    for (i = start_i; i < end_i; i++) {
        global_forces[i].fx = 0.0;
        global_forces[i].fy = 0.0;
        global_forces[i].fz = 0.0;

        for (t = 0; t < nthreads; t++) {
            global_forces[i].fx += thread_forces[t][i].fx;
            global_forces[i].fy += thread_forces[t][i].fy;
            global_forces[i].fz += thread_forces[t][i].fz;
        }
    }
}
```

4. Синхронизация и вывод (ожидание завершения объединения вычисленных сил всех частиц pthread_barrier_wait. При прохождении барьера главный поток последовательно записывает состояние текущего кадра (координат всех N тел) в CSV-файл)
5. Обновление состояний Update_particles (Пересчет скоростей и координат частиц по формулам Эйлера выполняется параллельно распределением индексов частиц по потокам)

```
void Update_particles(int start_i, int end_i) {
    for (int i = start_i; i < end_i; i++) {
        particles[i].x += particles[i].vx * time_step;
        particles[i].y += particles[i].vy * time_step;
        particles[i].z += particles[i].vz * time_step;

        particles[i].vx += (global_forces[i].fx / particles[i].m) * time_step;
        particles[i].vy += (global_forces[i].fy / particles[i].m) * time_step;
        particles[i].vz += (global_forces[i].fz / particles[i].m) * time_step;
    }
}
```

6. Синхронизация и переход на следующий шаг (Ожидание завершения обновления положения всех частиц и переход на следующий кадр времени)

3. Результаты замеров производительности

Сводная таблица производительности

Для замеров производительности используется скрипт benchmark.sh. Стоит упомянуть, что в запускаемом скрипте не учитывается время записи результатов расчета траекторий частиц в файл, так как это медленная операция и она не относится к алгоритму.

Тестирование проводилось на AMD Ryzen 5 5500U, ОС Ubuntu 22.04.5 LTS. Компиляция проходила с помощью gcc с флагом оптимизации -O3. Для каждого числа потоков (это 1, 2, 4, 6, 8) программа запускалась 5 раз, бралось среднее всех измерений.

В таблице представлены результаты экспериментов для системы из $N = 2000$ частиц при изменении числа рабочих потоков $p \in \{1,2,4,6,8\}$.

Количество потоков	Среднее время (с)	Ускорение	Эффективность
1	4.707540	1.000000	1.000000
2	3.362480	1.400020	0.700010
4	2.485440	1.894047	0.473512
6	2.095720	2.246264	0.374377
8	2.278320	2.066233	0.258279

Исходя из результатов измерений среднего времени выполнения программы и расчета ускорения и эффективности в зависимости от количества потоков, можно сделать вывод, что ускорение почти в 2 раза дает количество

потоков, большее 2, однако эффективность наоборот падает с ростом числа потоков.

Задание 3

1. Организация my_rwlock

Для реализации своего типа будем использовать предложенный подход, создадим следующую структуру:

• *Мьютекс*

• *Две условные переменные (одну для читателей, другую для писателей)*

• *Счетчик читателей (сколько потоков в данный момент читают)*

• *Счетчик количества потоков, ожидающих получения блокировки на чтение*

• *Счетчик количества потоков, ожидающих получения блокировки на запись*

• *Флаг, показывающий, получил ли блокировку хотя бы один писатель в данный момент.*

Сама блокировка - это набор счётчиков в структуре. Мьютекс держится лишь на время проверки и изменения этих счётчиков, а не всё время работы с данными.

Еще есть проблема выбора подхода – отдавать предпочтение читателям или писателям. По умолчанию в стандарте POSIX (pthread_rwlock_t) поведение не определено, и выбор приоритета зависит от ОС. Например, в Linux (а именно в ней мы работаем через WSL) это PTHREAD_RWLOCK_PREFER_READER_NP – приоритет читателей по умолчанию. Читатель ждёт только пока писатель держит блокировку. Если писатель только ждёт, новый читатель всё равно входит к уже читающим.

2. Реализация с помощью pthreads

Все функции строятся по одному принципу, сначала мы захватываем мьютекс, проверяем и изменяем счётчики, а потом освобождаем мьютекс. Как и функции pthreads, мы будем возвращать 0, если всё успешно, и 1 – при ошибке.

1. rwlock_init

```
int rwlock_init(rwlock_t* lock){
    int err;

    err = pthread_mutex_init(&lock->mutex, NULL);
    if(err != 0)
        return err;
    err = pthread_cond_init(&lock->readers_cond, NULL);
    if(err != 0){
        pthread_mutex_destroy(&lock->mutex);
        return err;
    }
    err = pthread_cond_init(&lock->writers_cond, NULL);
    if(err != 0){
        pthread_cond_destroy(&lock->readers_cond);
        pthread_mutex_destroy(&lock->mutex);
        return err;
    }

    lock->readers = 0;
    lock->readers_waiting = 0;
    lock->writers_waiting = 0;
    lock->writer = 0;
    return 0;
}
```

Здесь мы создаем мьютекс и две условные переменные и обнуляем счётчики. Если один из объектов не создался, уже созданные уничтожаются.

2. rwlock_destroy

```
int rwlock_destroy(rwlock_t* lock){
    int is_busy;

    pthread_mutex_lock(&lock->mutex);
    is_busy = lock->readers > 0 || lock->writer ||
           lock->readers_waiting > 0 || lock->writers_waiting > 0;
    pthread_mutex_unlock(&lock->mutex);
    if(is_busy)
        return EBUSY;

    pthread_cond_destroy(&lock->writers_cond);
    pthread_cond_destroy(&lock->readers_cond);
    pthread_mutex_destroy(&lock->mutex);
    return 0;
}
```

Здесь идёт проверка счётчиков (есть ли читатели/писатели или ожидающие чтения/записи). Если они есть, то возвращается код EBUSY, иначе всё успешно уничтожается.

3. rwlock_rdlock

```
int rwlock_rdlock(rwlock_t* lock){
    int err;

    err = pthread_mutex_lock(&lock->mutex);
    if(err != 0)
        return err;

    lock->readers_waiting++;
    while(lock->writer)
        pthread_cond_wait(&lock->readers_cond, &lock->mutex);
    lock->readers_waiting--;
    lock->readers++;

    pthread_mutex_unlock(&lock->mutex);
    return 0;
}
```

Здесь поток захватывает мьютекс и увеличивает счётчик ожидающих писателей, так unlock узнает, что есть ждущий читатель. Пока есть читатель, поток спит на readers_cond. pthread_cond_wait отпускает мьютекс на время сна и снова захватывает его при пробуждении. Если писателей нет, поток уходит из очереди, становится читателем (readers++) и отпускает мьютекс.

4. rwlock_wrlock

```
int rwlock_wrlock(rwlock_t* lock){
    int err;

    err = pthread_mutex_lock(&lock->mutex);
    if(err != 0)
        return err;

    lock->writers_waiting++;
    while(lock->writer || lock->readers > 0)
        pthread_cond_wait(&lock->writers_cond, &lock->mutex);
    lock->writers_waiting--;
    lock->writer = 1;

    pthread_mutex_unlock(&lock->mutex);
    return 0;
}
```

Писатель входит, только когда блокировка полностью свободна, нет другого писателя и ни одного читателя. После этого он ставит writer = 1, и до unlock никто другой не войдёт.

5. rwlock_unlock

```
int rwlock_unlock(rwlock_t* lock){
    int err;

    err = pthread_mutex_lock(&lock->mutex);
    if(err != 0)
        return err;

    if(lock->writer)
        lock->writer = 0;
    else if(lock->readers > 0)
        lock->readers--;
    else{
        pthread_mutex_unlock(&lock->mutex);
        return EPERM;
    }

    if(lock->readers == 0){
        if(lock->readers_waiting > 0)
            pthread_cond_broadcast(&lock->readers_cond);
        else if(lock->writers_waiting > 0)
            pthread_cond_signal(&lock->writers_cond);
    }

    pthread_mutex_unlock(&lock->mutex);
    return 0;
}
```

Если стоит флаг writer, то выходит писатель (т.к. читателей быть не может). Иначе выходит один из читателей. unlock незанятой блокировки возвращает EPERM, как мьютекс с проверкой ошибок.

Если блокировка освободилась полностью, сначала будятся читатели (причем всех, т.к. они могут читать параллельно), а писатель только если в очереди нет читателей.

3. Результаты замеров производительности

Сводная таблица производительности

Тестировалась реализация с помощью pth_ll_rwl.c, вставлялось 100 ключей, проводилось 100000 операций, 0.9 – чтения, 0.1 – на запись. Бралось среднее из 5 измерений.

Тестирование проводилось на Intel Core i7-12700H, ОС Ubuntu 26.04.1 LTS.

Компиляция проходила с помощью gcc с флагом оптимизации -O2.

Кол-во потоков	**Версия**	**Среднее время**	**Ускорение**	**Эффективность**
1 поток	Библиотечная реализация	0.7609	1.000	1.0000
	my_rwlock	1.2070	1.000	1.0000
4 потока	Библиотечная реализация	0.8454	0.900	0.2253
	my_rwlock	1.0821	1.115	0,2795
6 потоков	Библиотечная реализация	0.8365	0.910	0.1529
	my_rwlock	1.1255	1.072	0,1792
8 потоков	Библиотечная реализация	0.9729	0.782	0,0983
	my_rwlock	1.2609	0.957	0.1206

Библиотечная реализация работает быстрее кастомной (0.7609 сек против 1.2070 сек при 1 потоке, примерно в 1,6 раз меньше). POSIX-блокировки оптимизированы на уровне ядра Linux (futex) и в отсутствие конкуренции выполняются как быстрые атомарные операции в пространстве

пользователя, тогда как my_rwlock всегда тратит время на захват мьютекса и проверку счетчиков.

Теперь повторим эксперимент при 0.99 чтения и 0.01 записи.

Кол-во потоков	Версия	Среднее время	Ускорение	Эффективность
1 поток	Библиотечная реализация	0.0755	1.0000	1.0000
	my_rwlock	0.0837	1.0000	1.0000
4 потока	Библиотечная реализация	0.1841	0.4101	0.1025
	my_rwlock	0.1047	0.7994	0.1999
6 потоков	Библиотечная реализация	0.1591	0.4745	0.0791
	my_rwlock	0.1858	0.4505	0.0751
8 потоков	Библиотечная реализация	0.1477	0.5112	0.0639
	my_rwlock	0.1846	0.4534	0.0567

Здесь также можно увидеть, что при увеличении числа потоков выигрыша нет. Даже если 99% времени - это чтение которое может проходить параллельно, 1% записи требует абсолютно последовательного выполнения.